

Course: Compiler Design & Finite Automata Theory Lab

Design and Implementation of a Mini Compiler (partial) with Automata Component

Objective

Design and implement a mini compiler for a custom programming language, along with an automata-based regex module. The project should demonstrate concepts from:

- Lexical Analysis
- Syntax Analysis
- Semantic Analysis
- Finite Automata

Part A: Mini Compiler (Core)

Language Name: MiniLang

1. Keywords

int, float, if, else, while, print

2. Operators

+ - * / = < > <= >= == !=

3. Delimiters

; { } ()

Grammar (LL(1)-friendly)

Program $\rightarrow$ StmtList

StmtList $\rightarrow$ Stmt StmtList | ϵ

Stmt $\rightarrow$ DeclStmt | AssignStmt | IfStmt | WhileStmt | PrintStmt

DeclStmt $\rightarrow$ Type id ;

Type $\rightarrow$ int | float

AssignStmt $\rightarrow$ id = Expr ;

IfStmt $\rightarrow$ if (Cond) { StmtList } ElsePart

ElsePart $\rightarrow$ else { StmtList } | ϵ

WhileStmt $\rightarrow$ while (Cond) { StmtList }

PrintStmt $\rightarrow$ print (Expr) ;

Cond $\rightarrow$ Expr RelOp Expr

RelOp $\rightarrow$ < | > | <= | >= | == | !=

Expr $\rightarrow$ Term Expr'

Expr' $\rightarrow$ + Term Expr' | - Term Expr' | ϵ

Term $\rightarrow$ Factor Term'

Term' $\rightarrow$ * Factor Term' | / Factor Term' | ϵ

Factor $\rightarrow$ id | number | (Expr)

Required Features

1. Lexical Analyzer

- Tokenize:
 - Keywords

- Identifiers
- Numbers
- Operators
- Output: Token stream

2. Syntax Analyzer

- Implement recursive descent parser
- Validate program using given grammar
- Show parsing steps OR parse tree

3. Symbol Table

Store:

- Variable name
- Type

4. Semantic Analysis

Detect:

- Undeclared variables
- Duplicate declarations
- Type mismatch

Part B: Automata Component

Task

Implement:

1. Regular Expression $\rightarrow$ NFA (Thompson's Construction)
2. NFA $\rightarrow$ DFA Conversion
3. String Simulation

- Input: regex + string
- Output: Accepted / Rejected

Example

Regex:

a(b|c)*

Test strings:

abbb ✓

accc ✓

a ✓

Input / Output Examples

Sample Input Program

```
int a;  
int b;  
a = 5;  
b = a * 2 + 3;  
print(b);
```

Expected Token Output

```
[int] [id:a] [;]  
[int] [id:b] [;]  
[id:a] [=] [num:5] [;]  
[id:b] [=] [id:a] [*] [num:2] [+] [num:3] [;]  
[print] [(] [id:b] [)] [;]
```

Test Cases

Case 1: Undeclared Variable

`a = 10;`

Output:

Error: Variable 'a' not declared

Case 2: Duplicate Declaration

`int a;`

`int a;`

Output:

Error: Variable 'a' already declared

Case 3: Syntax Error

`int a`

`a = 5;`

Output:

Syntax Error: Missing ';'

Case 4: Type Mismatch

`int a;`

`a = 3.5;`

Output:

Type Error: Cannot assign float to int

Case 5: If-Else

```
int a;  
a = 10;  
if (a > 5) {  
    print(a);  
} else {  
    print(0);  
}
```

Case 6: While Loop

```
int a;  
a = 3;  
while (a > 0) {  
    print(a);  
    a = a - 1;  
}
```